ESTRUCTURA DE DATOS

PROF. GRISELDA ROMERO

UNIVERSIDAD AMERICANA

LISTAS EN C++

Introducción a las Listas en C++

Una lista es una estructura de datos dinámica que permite almacenar un conjunto de elementos de manera secuencial, pero con la ventaja de que su tamaño puede crecer o reducirse en tiempo de ejecución.
A diferencia de los vectores o matrices (que tienen tamaño fijo), las listas permiten:
- Insertar elementos en cualquier posición.
- Eliminar elementos sin necesidad de mover todos los demás.
- Recorrer los elementos de forma ordenada.

En C++ existen dos formas principales de trabajar con listas:
- Listas implementadas con punteros n(listas enlazadas).
- Listas de la STL (Standard Template Library), como std::list.

Tipos de listas enlazadas

- Lista simplemente enlazada: Cada nodo contiene un dato y un puntero al siguiente nodo.
- Lista doblemente enlazada: Cada nodo tiene punteros al anterior y al siguiente, lo que permite recorrer en ambos sentidos.
- Lista circular: El último nodo apunta al primero

Ejemplo básico – Insertar registros en una lista

main.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Alumno {
    int legajo;
    string nombre;
    int edad;
    Alumno* siguiente;
};

void insertarAlumno(Alumno*& cabeza, int legajo, string nombre, int edad) {
    Alumno* nuevo = new Alumno();
    nuevo->legajo = legajo;
    nuevo->nombre = nombre;
    nuevo->edad = edad;
    nuevo->siguiente = nullptr;

    if (cabeza == nullptr) {
        cabeza = nuevo;
    } else {
        Alumno* actual = cabeza;
        while (actual->siguiente != nullptr) {
            actual = actual->siguiente;
        }
        actual->siguiente = nuevo;
    }
}

void mostrarAlumnos(Alumno* cabeza) {
```

UNIVERSIDAD AMERICANA

```
30 void mostrarAlumnos(Alumno* cabeza) {
31     cout << "\n--- Lista de alumnos ---" << endl;
32     Alumno* actual = cabeza;
33     while (actual != nullptr) {
34         cout << "Legajo: " << actual->legajo
35              << " | Nombre: " << actual->nombre
36              << " | Edad: " << actual->edad << endl;
37         actual = actual->siguiente;
38     }
39 }
40
41 int main() {
42     Alumno* lista = nullptr;
43     int n;
44
45     cout << "¿Cuantos alumnos desea registrar? ";
46     cin >> n;
47     cin.ignore(); // limpiar buffer después de leer número
48
49     for (int i = 0; i < n; i++) {
50         int legajo, edad;
51         string nombre;
52
53         cout << "\nAlumno " << i+1 << endl;
54         cout << "Legajo: ";
55         cin >> legajo;
56         cin.ignore(); // limpiar buffer antes de getline
57
58         cout << "Nombre: ";
59         getline(cin, nombre);
```

```
58          cout << "Nombre: ";
59          getline(cin, nombre);
60
61          cout << "Edad: ";
62          cin >> edad;
63          cin.ignore(); // limpiar buffer
64
65          insertarAlumno(lista, legajo, nombre, edad);
66      }
67
68      mostrarAlumnos(lista);
69
70      return 0;
71  }
72
```

Resultado de la ejecución :

```
¿Cuantos alumnos desea registrar? 2

Alumno 1
Legajo: 100
Nombre: ana
Edad: 20

Alumno 2
Legajo: 101
Nombre: pepe
Edad: 19

--- Lista de alumnos ---
Legajo: 100 | Nombre: ana | Edad: 20
Legajo: 101 | Nombre: pepe | Edad: 19
```

Explicaciones :

```
5  struct Alumno {
6      int legajo;
7      string nombre;
8      int edad;
9      Alumno* siguiente;
10 };
11
```

Se define un tipo de dato personalizado llamado Alumno.
Contiene tres campos de información: legajo, nombre y edad.
Además, tiene un puntero siguiente que conecta un alumno con el próximo en la lista (esto es lo que convierte la estructura en una lista enlazada).

```
12 void insertarAlumno(Alumno*& cabeza, int legajo, string nombre, int edad) {
13     Alumno* nuevo = new Alumno();
14     nuevo->legajo = legajo;
15     nuevo->nombre = nombre;
16     nuevo->edad = edad;
17     nuevo->siguiente = nullptr;
```

Se crea dinámicamente un nuevo nodo (new Alumno()).
Se cargan los datos ingresados por el usuario.
nuevo->siguiente = nullptr indica que será el último nodo de la lista.

```
19      if (cabeza == nullptr) {
20          cabeza = nuevo;
21      } else {
22          Alumno* actual = cabeza;
23          while (actual->siguiente != nullptr) {
24              actual = actual->siguiente;
25          }
26          actual->siguiente = nuevo;
27      }
28  }
29
```

Si la lista está vacía (cabeza == nullptr), el nuevo alumno pasa a ser el primero.
Si ya hay alumnos, se recorre la lista hasta el último nodo y se enlaza el nuevo al final.

```
29
30  void mostrarAlumnos(Alumno* cabeza) {
31      cout << "\n--- Lista de alumnos ---" << endl;
32      Alumno* actual = cabeza;
33      while (actual != nullptr) {
34          cout << "Legajo: " << actual->legajo
35               << " | Nombre: " << actual->nombre
36               << " | Edad: " << actual->edad << endl;
37          actual = actual->siguiente;
38      }
39  }
40
```

Recorre la lista desde el primer nodo (cabeza) hasta que actual sea nullptr.
En cada paso imprime los datos del alumno.
Esto muestra la lista completa en pantalla.

```
40
41  int main() {
42      Alumno* lista = nullptr;
43      int n;
44
45      cout << "¿Cuantos alumnos desea registrar? ";
46      cin >> n;
47      cin.ignore(); // limpiar buffer después de leer número
48
```

Se declara la lista vacía (lista = nullptr).
Se pregunta cuántos alumnos se van a registrar.
cin.ignore() limpia el buffer para evitar problemas al usar getline.cpp

```
49      for (int i = 0; i < n; i++) {
50          int legajo, edad;
51          string nombre;
52
53          cout << "\nAlumno " << i+1 << endl;
54          cout << "Legajo: ";
55          cin >> legajo;
56          cin.ignore(); // limpiar buffer antes de getline
57
58          cout << "Nombre: ";
59          getline(cin, nombre);
60
61          cout << "Edad: ";
62          cin >> edad;
63          cin.ignore(); // limpiar buffer
64
65          insertarAlumno(lista, legajo, nombre, edad);
66      }
```

Se repite el proceso de carga de datos n veces.
Se pide legajo, nombre y edad.
Cada alumno se inserta en la lista con insertarAlumno.

```
68      mostrarAlumnos(lista);
69
70      return 0;
71  }
72
```

Finalmente se llama a mostrarAlumnos para imprimir la lista completa.

Con este programa se aprende :

- Cómo se construye una lista enlazada paso a paso.
- Cómo se insertan nodos dinámicamente con new.
- Cómo se recorre la lista con un puntero auxiliar.
- Cómo se integran entrada de datos del usuario con estructuras dinámicas.

Actividad de proceso de clase :

Selecciona un tema de entre las opciones de abajo y luego escribe el planteamiento y la solución

Opción 1 : clínica

Opción 2 : banco

Opción 3 : comercio